

计算机程序设计基础(2)

--- C++程序设计(8)

孙甲松

sunjiasong@tsinghua.edu.cn

电子工程系 信息认知与智能系统研究所

罗姆楼6-104

电话: 62796193/13901216180

2020.4.

1

6. 模板

模板 (template) 是C++支持参数化多态性的工具, 是将一段程序所处理对象的类型参数化, 这样各种类型的对象都可以用同一段程序处理, 省去重复编写程序的麻烦, 这称为程序复用或代码复用 (code re-used)。

这样的一段把类型参数化的程序被称为一个模板。

C++模板主要针对函数和类等, 形成:

函数模板、类模板、数组类模板

语法形式为:

template < 模板参数表 > 声明

2

6. 模板（续1）

模板参数表 可以为：

class 标识符

typename 标识符

这里：标识符 可以是类类型或数据类型

声明 指函数或类的说明或定义

例如：template <class T> 或：template <typename T>

T abs(T a)

T abs(T a)

{ return a<0 ? -a: a;}

{ return a<0 ? -a: a;}

这里的abs是一个求变量a绝对值的函数模板，只要变量a可以和0比较大小，都可以使用这个函数模板自动生成求绝对值的函数。

3

目 录

6.1 函数模板

6.1.1 函数模板实参的使用

6.1.2 函数模板中的常量参数化

6.1.3 函数模板中的静态对象

6.1.4 冒泡排序函数模板

6.2 类模板

6.2.1 类模板中的常量参数化

6.3 数组类模板

4

6.1 函数模板

- 函数模板是为了避免实现相同功能而不得不书写代码几乎完全相同的多个重载函数。
- 编译系统会根据实际调用情况，由函数模板生成重载的模板函数实例。
- 使用函数模板只是简化程序的代码书写，并不能提高程序的执行效率。
- 如果一个函数模板声明定义中有局部的静态对象，那么这个静态对象也会在每个模板函数中被定义，各模板函数中的静态变量是相互独立的。

5

6.1 函数模板

函数模板的一般形式：

```
template <class T>  
函数返回类型 函数模板名(形参表)  
{函数体}
```

或：

```
template <typename T>  
函数返回类型 函数模板名(形参表)  
{函数体}
```

通常把形参表的参数类型或者函数返回类型定义为参数化的T，使这个模板程序对各种类型都适用。

6

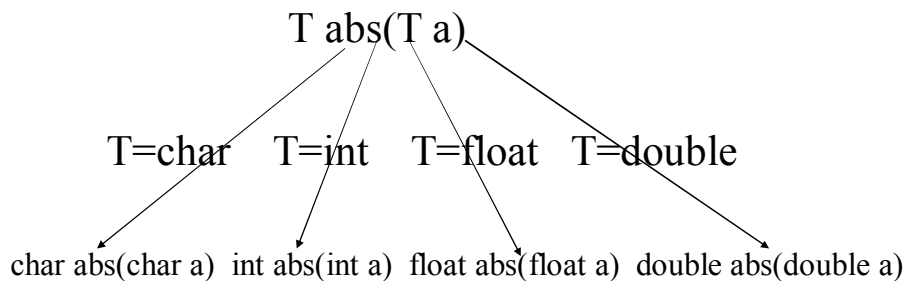
6.1 函数模板

```
#include <iostream>
using namespace std;
template <typename T>
T abs(T a)
{ return a<0 ? -a : a;
}
void main( )
{ char c=-0x63;    int d=-32767;
  float e=-1.2345f; double f=-1.234e10;
  cout <<abs(c) <<' ' <<abs(d) <<' ';
  cout <<abs(e) <<' ' <<abs(f) << endl;
}
```

7

6.1 函数模板(续1)

编译时，由abs(c)、abs(d)、abs(e)、abs(f)这四个函数调用，自动生成四个abs函数模板的实例，它们之间是重载的函数。



下面是一个有与函数模板同名的函数的例子：

8

```

#include <iostream>
using namespace std;
template <class T>
T min1(T a, T b)
{ return a<b ? a : b; }
int min1(int a, int b)
{ return a<b ? a : b; }
float min1(int a, float b)
{ return a<b ? a : b; }
void main( )
{ int a=-3, b=5; float e=1.25f, f=-5.21f;
  double d1=1.25e-10, d2=5.21e10;
  cout <<min1(a,b) <<' ' <<min1(e,f) <<' ';
  cout <<min1(d1, d2)<<endl;
  cout <<min1(a,f) <<endl;    // int 和float
}

```

9

编译无错误，运行结果：

-3 -5.21 1.25e-010

-5.21

请按任意键继续...

如果把主函数的最后一行：

cout <<min1(a, f) <<endl; // int和float

改为：

cout <<min1(f, a) <<endl; // float和int

编译结果：

test.cpp(15) : warning C4244:

“参数”：从“float”转换到“int”，可能丢失数据

编译无错误，运行结果：

-3 -5.21 1.25e-010

-5

请按任意键继续...

10

6.1 函数模板(续2)

有函数模板 `T min1(T a, T b)`，同时还有同名的重载函数

`int min1(int a, int b)` 和 `float min1(int a, float b)`

1. 编译系统根据参数类型，首先在 `min1` 重载函数中进行参数类型匹配，找到则调用，否则，转2；

`min1(a, f)` 参数类型是 `int` 和 `float`，调用 `float min1(int a, float b)` 重载函数

`min1(a, b)` 参数类型是 `int` 和 `int`，调用 `int min1(int a, int b)` 重载函数

2. 试图用函数模板生成一个 `min1` 模板函数，若能生成则OK；

`min1(e, f)` 参数类型是 `float` 和 `float`，使 `T=float`，生成

`float min1(float a, float b)`

`min1(d1, d2)` 参数是 `double` 和 `double`，使 `T=double`，生成

`double min1(double a, double b)`

11

3. 若不能确定 `T` 的值，编译器将通过类型强制转换试图使函数调用和其中一个已经存在的重载函数匹配。这一步许多编译系统不做，以前的VC6不做，但VS2008做；

`min1(f, a)` 参数是 `float` 和 `int`，通过强制类型转换与

`int min1(int a, int b)` 匹配，但因为 `float` 转换为 `int` 会丢失数据精度，编译系统给出警告性错误信息。

如果 `int min1(int a, int b)` 重载函数不存在，将通过强制类型转换与 `float min1(int a, float b)` 匹配，但因为 `float` 转换为 `int`、`int` 转换为 `float` 都会丢失数据精度，编译系统给出两次警告性错误信息。

4. 如果通过类型强制转换也不能匹配，系统给出错误信息。

如果把 `int min1(int a, int b)` 和 `float min1(int a, float b)` 重载函数都去掉，`min1(a, f)` 或 `min1(f, a)` 编译时都会产生错误：

error C2782: “`T min1(T, T)`”: 模板参数 “`T`” 不明确

可能是 “`float`” 或 “`int`”

编译器不知道应该把 `int` 转换为 `float`，使得 `T=float`；还是应该把 `float` 转换为 `int`，使得 `T=int`，存在不确定性，所以出错。

12

6.1 函数模板(续3)

当多个类型参数化时，编译系统将对实参的类型进行静态类型分析，来确定函数模板中对应位置的类型参数的类型。

```
#include <iostream>
using namespace std;
template <class X, class Y>
Y fun(X x, Y y)
{ return x+y; }
void main( )
{ double a=1.0; short b=12;
  cout << fun(a, b); // a使得X为double, b使得Y为short
}
```

所以模板参数必须出现在函数声明的形参表中。

13

程序：

```
#include <iostream>
using namespace std;
template <class X, class Y, class Z >
Z fun(X x, Y y)
{ return x+y; }
void main( )
{ double a=1.0; short b=12;
  cout << fun(a, b); // a使得X为double, b使得Y为short
}
```

是错误的，编译系统无法根据fun实参a, b的类型来确定Z的类型，编译结果：

```
test.cpp(8) : error C2783: "Z fun(X,Y)": 无法为"Z"推导模板参数
test.cpp(4) : 参见"fun"的声明
```

14

同理，程序：

```
#include <iostream>
using namespace std;
template <class T>
T fun( )
{ return 1; }
void main( )
{
    cout << fun( );
}
```

也是错误的，由于调用时没有实参，编译系统无法根据实参类型来确定T的值，编译结果：

```
test.cpp(8) : error C2783: "T fun(void)": 无法为"T"推导模板参数
test.cpp(4) : 参见"fun"的声明
```

15

程序：

```
#include <iostream>
using namespace std;
template <class X, class Y>
Y max1(X a, Y b)
{ return a>b ? a : b; // 第5行
}
void main( )
{ int    a=-3, b=-25;
  float  e=1.25f, f=-15.71f;
  cout << max1(b,f) << endl; // int->X, float->Y, 返回Y
  cout << max1(f,b) << endl; // float->X, int->Y, 返回Y
}
```

编译时会有警告：

```
test.cpp(5) : warning C4244: "return": 从"float"转换到"int"，可能丢失数据
```

16

6.1.1 函数模板实参的使用

由于会出现编译系统无法根据调用函数模板时的实参类型来确定模板参数的类型，所以C++近些年引入了函数模板实参的概念。

方法是：在调用函数模板时，单独用函数模板实参来说明模板参数的类型。例如对函数模板fun：

```
template <class X, class Y, class Z>
Z fun(X x, Y y)
{ return x+y; }
```

调用格式是： fun<int, long, double>(a, b)

对应强制给 < X, Y, Z>赋值，使得：

X=int, Y=long, Z=double

从而避免出现编译系统无法根据实参来确定Z的类型。

17

```
#include <iostream>
using namespace std;
template <typename T>
T min1(T a, T b)
{ return a<b ? a : b; }
template <typename X, typename Y>
X max1(X a, Y b)
{ return a>b ? a : b; } //第8行
void main( )
{ int b=-25; float f=-15.71f;
  cout<<min1(3,5.0)<<endl; // 第11行
  cout<<max1(f, b) <<endl; // float->X, int->Y,返回float
  cout<<max1<int>(f, b)<<endl; // int->X, int->Y,返回int 第13行
  cout<<max1<int,float>(f, b)<<endl; //int->X, float->Y,返回int 第14行
}
```

18

编译结果：

test.cpp(11): error C2782: "T min1(T,T)": 模板参数"T"不明确

test.cpp(4) : 参见"min1"的声明

可能是"double"

或 "int"

test.cpp(13) : warning C4244: "参数" : 从 "float"转换到 "int", 可能丢失数据

test.cpp(14) : warning C4244: "参数" : 从 "int"转换到 "float", 可能丢失数据

test.cpp(14) : warning C4244: "参数" : 从 "float"转换到 "int", 可能丢失数据

1>test - 1 个错误, 3个警告

出现错误的原因是：函数调用min1(3, 5.0),

参数类型是 (int, double), 这使得编译器无法确定T到底应该是int还是 double, 因而出错, 除非自己用函数模板实参指定T的类型:

min1<double>(3,5.0)

修改后:

19

```
#include <iostream>
using namespace std;
template <typename T>
T min1(T a, T b)
{ return a<b ? a : b; }
template <typename X, typename Y>
X max1(X a, Y b)
{ return a>b ? a : b; } //第8行
void main()
{ int b=-25; float f=-15.71f;
  cout<<min1<double>(3, 5.0)<<endl; // 第11行
  cout<<max1(f, b) <<endl; // float->X, int->Y,返回float
  cout<<max1<int>(f, b)<<endl; // int->X, int->Y,返回int 第13行
  cout<<max1<int, float>(f, b)<<endl; // int->X,float->Y,返回int 第14行
}
```

20

编译结果:

```
1>test.cpp
1>test.cpp(13): warning C4244: "参数": 从 "float"转换到 "int", 可能丢失数据
1>test.cpp(14): warning C4244: "参数": 从 "int"转换到 "float", 可能丢失数据
1>test.cpp(14): warning C4244: "参数": 从 "float"转换到 "int", 可能丢失数据
1>test.cpp(8): warning C4244: "return": 从 "float"转换到 "int", 可能丢失数据
1> test.cpp(14): 参见对正在编译的函数模板实例化 "X max1<int,float>(X,Y)"的引用
1> with
1> [
1>     X=int,
1>     Y=float
1> ]
1>test - 0 个错误, 4 个警告
```

运行结果:

```
5
-15.71
-15
-15
请按任意键继续...
```

21

```
#include <iostream>
using namespace std;
template <typename T>
T data( )
{ return 20;
}
template <typename X, typename Y, typename Z>
Z max2(X a, Y b)
{ return a>b ? a : b;
}
void main( )
{ int b=-25; float f=-15.71f;
  cout << max2<int, float, float>(f, b) <<endl; // line 11
           // int->X, float->Y, float->Z
  cout << data<float>( ) <<endl; // float->T
}
```

22

编译结果:

```
test.cpp(11) : warning C4244: "参数" :  
             从"int"转换到"float", 可能丢失数据  
test.cpp(11) : warning C4244: "参数" :  
             从"float"转换到"int", 可能丢失数据  
1>test - 0 个错误, 2个警告
```

运行结果:

-15

20

请按任意键继续...

23

6.1.2 函数模板中的常量参数化

在函数模板中, 可以用一般修饰符说明的常规参数, 对函数模板中所用到的常量进行参数化, 从而使得用二维及二维以上数组作参数的函数能通用化。

```
#include <iostream>  
using namespace std;  
template <typename T, int Rows, int Cols>  
void sumRow(T data[ ][Cols], T result[ ])   
{  int i,j;  
   for (i=0; i<Rows; i++)  
   {  result[i]=0;  
     for (j=0; j<Cols; j++)  
       result[i] += data[i][j];  
   }  
}
```

24

```

void main( )
{
    int a[ ][3]={1,2,3,4,5,6,7,8,9,0, 1, 2};
    int r[4], i;
    sumRow<int,4,3>(a, r);
    for (i=0; i<4; i++)
        cout <<r[i]<<' ';
    cout<<endl;
    double d[ ][4]={1,2,3,4,5,6,7,8,9,0, 1, 2};
    double r2[3];
    sumRow<double,3,4>(d, r2);
    for (i=0; i<3; i++)
        cout <<r2[i]<<' ';
    cout<<endl;
}

```

运行结果:

```

6 15 24 3
10 26 12
请按任意键继续. . .

```

25

6.1.3 函数模板中的静态对象

如果一个函数模板声明中定义有局部的静态对象，那么这个静态对象也将在每个模板函数中被定义，各模板函数之中的静态变量是相互独立的。

```

#include <iostream>
using namespace std;
template <typename T>
T abs2(T a)
{
    static int cc=0;
    cout << "cc=" << ++cc << ' ' << a << endl;
    return a<0 ? -a : a;
}

void main( )
{
    int c=-63, d=-32767;
    double e=-1.2345, f=-1.234e10;
    c=abs2(c); e=abs2(e); d=abs2(d); f=abs2(f);
    cout << c << ' ' << d << ' ' << e << ' ' << f << endl;
}

```



从结果可以看出，int abs2(int a) 和double abs2(double a)这两个模板实例的静态变量cc是不相关的

26

6.1.4 冒泡排序函数模板

我们通过把C语言中讲过的冒泡排序程序改造为函数模板，说明写函数模板的好处。

C语言冒泡排序程序：

```
void bubblesort(ET p[ ], int n)
{   int m, k, j, i;
    ET d;
    k=0; m=n-1;
    while (k<m)
    {   ..... }
    return;
}
```

涉及到数据类型只有ET p[] 和ET d; 我们说过ET可以是任意C允许的数据类型，这里我们把它变成函数模板的类型参量T，程序可以改为：

27

```
#include <iostream>
using namespace std;
template <class ET> // 也可写为 template <typename ET>
void bubblesort(ET p[ ], int n)
{   int m, k, j, i;
    ET d; // 用来交换的中间变量
    k=0; m=n-1;
    while (k<m) // 子表未空
    {   j=m-1; m=0;
        for (i=k; i<=j; i++) // 从前往后扫描子表
            if (p[i]>p[i+1]) // 发现逆序进行交换
                { d=p[i]; p[i]=p[i+1]; p[i+1]=d; m=i; }
        j=k+1; k=0;
        for (i=m; i>=j; i--) // 从后往前扫描子表
            if (p[i-1] >p[i]) // 发现逆序进行交换
                { d=p[i]; p[i]=p[i-1]; p[i-1]=d; k=i; }
    }
    return;
}
```

28

```

void main( )
{ int a[10]={5,4,2,3,1,10,9,7,8,6}, i;
  double d[9]={1.2, 3.4, 2.3, 9.8, 6.7, 5.6, 4.5, 7.8, 8.9};
  for (i=0; i<10; i++)
    cout << a[i] << ' ';
  cout << endl;
  bubblesort(a, 10);
  for (i=0; i<10; i++)
    cout << a[i] << ' ';
  cout << endl;
  for (int i=0; i<9; i++)
    cout << d[i] << ' ';
  cout << endl;
  bubblesort(d, 9);
  for (int i=0; i<9; i++)
    cout << d[i] << ' ';
  cout << endl;
}

```

```

C:\WINDOWS.0\system32\cm...
5 4 2 3 1 10 9 7 8 6
1 2 3 4 5 6 7 8 9 10
1.2 3.4 2.3 9.8 6.7 5.6 4.5 7.8 8.9
1.2 2.3 3.4 4.5 5.6 6.7 7.8 8.9 9.8
请按任意键继续. . .

```

我们可以看出，把冒泡排序函数改成函数模板后，我们可以像调用普通函数一样调用，分别对不同数据类型的数组排序，快捷方便。

29

6.2 类模板

- 类模板使实现类程序中所需要的数据类型参数化。
- 适合于：包容性质的类
- 包容性的类：其对象很象一个容器，操作特性与对象类型无关或很少有关系。
- 栈、队列等是典型的包容性质的类，适合做成类模板。
- 语法形式是：

```

template <模板参数表>
class 类名
{类成员声明}

```

- 成员函数若要在类模板外定义，需要写成下面的形式：

```

template <模板参数表>
类型名 类名 <T>::成员函数名(形参表)

```

30

6.2 类模板

模板参数表 可写作：

class 标识符

typename 标识符

这里：标识符可以是数据类型或类类型

注意：模板类的每个成员函数都必须是函数模板。

用类模板来建立对象，应该按照如下的形式：

类模板名 <模板参数表> 对象1，对象2，.....;

通过把参数绑定到形式参数就可以创建具体的类，

这个过程称为类模板的实例化。

例如：栈的类模板

31

6.2 类模板（续1）

```
#include <iostream>
using namespace std;
template <class T>
class Stack {    // 栈的类模板
public: Stack(int size);
        Stack(const Stack &p);
        virtual ~Stack( );
        void Push(const T &e);
        const T &Pop( );
        const T &Peek( ) const;
        bool IsEmpty( ) const;
private: T *buff;
        int max;
        int top;
};
```

32


```

template <class T>
Stack <T>::Stack(int size) // 栈类模板的构造函数模板
{ buff = new T[max=size];
  top = -1;
}
template <class T>
Stack <T>::Stack(const Stack &p) // 栈类模板的复制构造函数模板
{ buff = new T[max=p.max];
  top = p.top;
  for (int i=0; i<=top;i++)
    buff[i] = p.buff[i];
}
template <class T>
Stack <T>::~~Stack( ) // 栈类模板的析构造函数模板
{ delete [ ]buff;
}

```

33

```

template <class T>
void Stack <T>::Push(const T &e) // 入栈操作函数模板
{ if (top >= max-1)
  cout<<"overflow!"<<endl;
  else
    buff[++top] = e;
}
template <class T>
const T & Stack <T>::Pop( ) // 出栈操作函数模板
{ if (top < 0)
  { cout<<"underflow!"<<endl;
    return buff[0]; // 这个返回值没有意义,仅为了通过编译
  }
  else
    return buff[top--];
}

```

34

```

template <class T>
const T & Stack <T>::Peek( ) const // 读栈顶元素函数模板
{ if (top < 0)
    { cout<<"underflow!"<<endl;
      return buff[0]; // 这个返回值没有意义,仅为了通过编译
    }
    else
        return buff[top];
}
template <class T>
bool Stack <T>::IsEmpty( ) const // 判断栈是否为空函数模板
{ return top < 0;
}
void main( )
{ Stack <double> DoubleStack(10); // T= double, size=10
  Stack <int> IntStack(15);        // T= int, size=15
  for (double i=1.0; i<=20.0; i+=2.0)
      DoubleStack.Push(i);
}

```

35

```

Stack <double> DoubleStack2(DoubleStack); // 先复制整个栈
while (!DoubleStack.IsEmpty( ))          // 再出栈
    cout << DoubleStack.Pop( ) << ", ";
cout << endl;
for (int j=0; j<75; j+=5)
    IntStack.Push(j);
Stack <int> IntStack2(IntStack); // 先复制整个栈
while (!IntStack.IsEmpty( ))      // 再出栈
    cout << IntStack.Pop( ) << ", ";
cout << endl;
while (!DoubleStack2.IsEmpty( ))
    cout << DoubleStack2.Pop( ) << ", ";
cout << endl;
while (!IntStack2.IsEmpty( ))
    cout << IntStack2.Pop( ) << ", ";
cout << endl;
}

```

36

6.2 类模板（续2）

编译系统根据下面的语句：

```
Stack <double> DoubleStack(10);
```

```
Stack <int> IntStack(15);
```

将生成double型的Stack对象DoubleStack，其栈的大小为10；生成int型的Stack对象IntStack，其栈的大小为15。

```
Stack <double> DoubleStack2(DoubleStack);
```

```
Stack <int> IntStack2(IntStack);
```

调用类模板Stack的复制构造函数生成一个与DoubleStack对象大小和内容完全相同的DoubleStack2对象，生成一个与IntStack对象大小和内容完全相同的IntStack2对象。

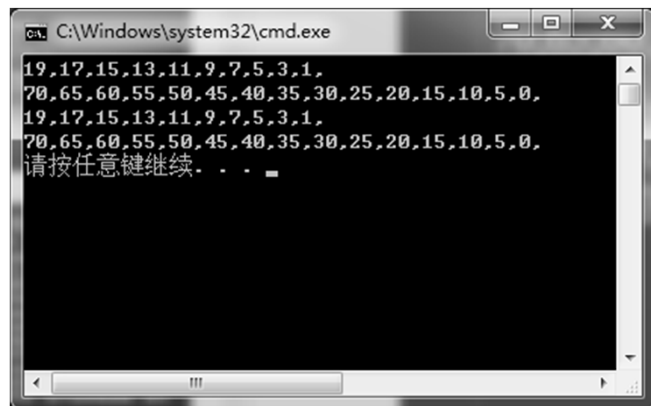
注意：为了使复制构造的对象与原对象内容完全相同，必须在被复制对象出栈之前进行复制构造。

每个栈对象各自包含相应double和int类型的成员和成员函数，这些函数虽然同名，但不是重载函数，因为他们隶属于不同的栈对象。

运行结果：

37

```
19,17,15,13,11,9,7,5,3,1,
70,65,60,55,50,45,40,35,30,25,20,15,10,5,0,
19,17,15,13,11,9,7,5,3,1,
70,65,60,55,50,45,40,35,30,25,20,15,10,5,0,
请按任意键继续...
```



38

如果主程序改为:

```
void main()  
{ Stack <double> DoubleStack(10);    // T= double, size=10  
  Stack <int> IntStack(15);          // T= int, size=15  
  for (double i=1.0; i<=21.0; i+=2.0)  
    DoubleStack.Push(i); //入栈数据量超过栈容量  
  Stack <double> DoubleStack2(DoubleStack);  
  while (!DoubleStack.IsEmpty())  
    cout << DoubleStack.Pop() << ", ";  
  cout << endl;  
  cout << DoubleStack.Pop() << endl; //栈空后再出栈一次  
  for (int j=0; j<=75; j+=5) //入栈数据量超过栈容量  
    IntStack.Push(j);  
  Stack <int> IntStack2(IntStack);  
  while (!IntStack.IsEmpty())  
    cout << IntStack.Pop() << ", ";  
  cout << endl;  
  cout << IntStack.Pop() << endl; //栈空后再出栈一次  
  while (!DoubleStack2.IsEmpty())  
    cout << DoubleStack2.Pop() << ", ";  
  cout << endl;  
  while (!IntStack2.IsEmpty())  
    cout << IntStack2.Pop() << ", ";  
  cout << endl;  
}
```

39

运行结果:

overflow!

19,17,15,13,11,9,7,5,3,1,

underflow!

1

overflow!

70,65,60,55,50,45,40,35,30,25,20,15,10,5,0,

underflow!

0

19,17,15,13,11,9,7,5,3,1,

70,65,60,55,50,45,40,35,30,25,20,15,10,5,0,

请按任意键继续...

40

6.2.1 类模板中的常量参数化

在类模板中，可以对常量进行参数化，这称为类模板的常量参数化。 例如：

```
template <class T, int N>
class Array {
    public: T &operator[ ](int i);
    private: T buf[N];
};
template <class T, int N> // 重载[]操作符函数模板
T & Array<T, N>::operator[ ](int i)
{    return buf[i]; }
```

41

6.2.1 类模板中的常量参数化

其中，参数模板N是参数化的常量，所参数化的是一个int常量，该常量在声明模板类时指定。

Array<int, 20> a;

从类模板Array<T,N>中生成模板类Array<int,20> 该模板类被用于声明对象a。在编译时生成静态的int buf[20]; 而前一个栈的例子的buff内存是按Stack <int> IntStack(15); 中的初值在程序运行时动态分配的。

42

```

#include<iostream>
using namespace std;
template <class T, int N>
class Array {
    public: T &operator[ ](int i);
    private: T buf[N];
};
template <class T, int N> // 重载[]操作符函数模板
T & Array<T, N>::operator[ ](int i)
{   return buf[i]; }

```

43

```

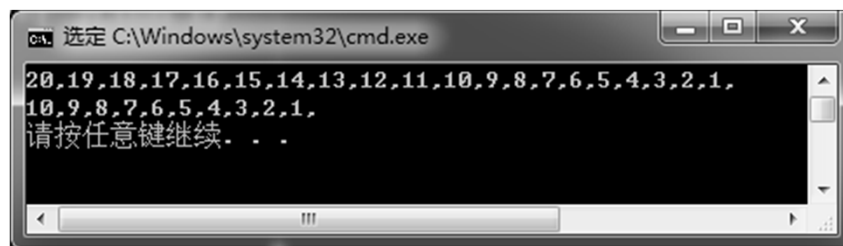
void main( )
{   Array<int, 20> a;
    int i;
    for (i=0;i<20; i++)
        a[i] = i+1;
    for (i=19;i>=0; i--)
        cout << a[i] << ",";
    cout << endl;
    Array<double, 10> b;
    for (i=0;i<10; i++)
        b[i] = i+1.0;
    for (i=9;i>=0; i--)
        cout << b[i] << ",";
    cout << endl;
}

```

44

运行结果：

20,19,18,17,16,15,14,13,12,11,10,9,8,7,6,5,4,3,2,1,
10,9,8,7,6,5,4,3,2,1,
请按任意键继续...



45

6.3 数组类模板

静态数组可通过下标直接访问其元素，但缺点是：

1. 数组长度必须是常数，编译时已经确定，运行时不能根据实际需要改变其长度，所以为了适用各种情况，会事先开大数组，这太浪费资源。
2. 静态数组不检查数组是否越界，所以不能避免下标越界引起的溢出产生致命错误的问题。

我们下面来设计一个自动检查是否下标越界、自动扩展内存长度的动态数组类模板Array。

46

```

#include <iostream>
#include <iomanip>
#include <cstdlib>
using namespace std;

typedef int ErrorType;
#define   invalidArraySize      0
#define   memoryAllocationError 1
#define   indexOutOfRange       2
char *errorMsg[ ]={"Invalid Array Size",
                  "Memory Allocation Error",
                  "Index Out Of Range"};

```

47

```

//数组类模板声明
template <class T>
class Array
{ public:
    Array(int sz = 50);           //构造函数
    Array(const Array<T>& A);      //复制构造函数
    ~Array(void);                //析构函数
    Array<T>& operator= (const Array<T>& rhs);
                                //重载"="使数组对象可以整体赋值
    T& operator[ ](int i);
                                //重载"[ ]", 使Array对象可以像C++普通数组使用
    operator T* (void) const;
                                //重载 "T*", 使Array对象可以起到C++普通数组的作用
    int ListSize(void) const;    // 获得当前数组的大小
    void Resize(int sz);         // 修改数组的大小
private:
    int size; //数组大小（元素个数）
    T *alist; //T类型指针，用于存放动态分配的内存首地址
    void Error(ErrorType error,int badIndex=0) const; // 错误处理
};

```

48


```

//构造函数
template <class T>
Array<T>::Array(int sz)
{
    if (sz<=0) //sz为数组大小(元素个数),若小于0,则输出错误信息
        Error(invalidArraySize);
    size = sz; // 将元素个数赋值给变量size
    alist = new T[size]; //动态分配size个T类型的元素空间
    if (alist == NULL) //如果分配内存不成功, 输出错误信息
        Error(memoryAllocationError);
}

//析构函数
template <class T>
Array<T>::~~Array(void)
{
    delete [ ]alist;
}

```

49

```

//复制构造函数
template <class T>
Array<T>::Array(const Array<T> &X)
{
    //从对象X取得数组大小, 并赋值给当前对象的成员
    int n = X.size;
    size = n;
    //为对象申请内存并进行出错检查
    alist = new T[n]; // 动态申请n个T类型的元素内存
    if (alist == NULL) //如果申请内存不成功, 输出错误信息
        Error(memoryAllocationError);
    //从对象X复制数组元素到本对象
    T *srcptr = X.alist; // X.alist是对象X的数组首地址
    T *destptr = alist; // alist是当前对象的数组首地址
    while (n--) // 逐个复制数组元素
        *destptr++ = *srcptr++;
}

```

50

```

// 重载“=”运算符，实现两个对象之间的整体赋值
template <class T>
Array<T>& Array<T>::operator= (const Array<T>& rhs)
{   int n = rhs.size;    // 取rhs的数组大小
    // 如果当前对象中数组大小与rhs不同，则删除本对象数组
    // 原有内存，然后重新分配内存
    if (size != n)
    {   delete [ ] alist;    // 删除数组原有内存
        alist = new T[n]; // 重新分配n个元素的内存
        if (alist == NULL) //如果分配内存不成功，输出错误信息
            Error(memoryAllocationError);
        size = n; //记录本对象的数组大小
    }
    T *destptr = alist; // 从rhs向本对象复制元素
    T *srcptr = rhs.alist;
    while (n--)
        *destptr++ = *srcptr++;
    return *this; // 返回当前对象的引用，解决 a=b=c 的问题
}

```

51

```

// 重载下标运算符[ ]，实现与普通数组一样通过下标访问元素
// 并且具有下标越界检查功能
template <class T>
T& Array<T>::operator[ ] (int n)
{
    if (n < 0 || n > size-1)    // 检查下标是否越界
        Error(indexOutOfRange,n);
    return alist[n]; // 返回当前对象中下标为n的alist数组元素
}
//重载指针转换运算符,将Array类的对象名转换为T类型的指针,
//指向当前对象中的私有数组。
//因而可以象使用普通数组首地址一样使用Array类的对象名
template <class T>
Array<T>::operator T * (void) const
{   return alist;    // 返回当前对象中私有数组的首地址
}

```

52

```

template <class T>
void Array<T>::Resize(int sz) // 将数组大小修改为sz
{ if (sz <= 0) // 检查是否sz<= 0
    Error(invalidArraySize);
  if (sz == size) return; // 若与原大小一样,什么也不做
  T *newlist = new T[sz]; // 申请新的数组内存
  if (newlist == NULL) // 测试申请内存是否申请成功
    Error(memoryAllocationError);
  int n=(sz<=size) ? sz : size; // 将sz与size中较小的一个赋值给n
  // 将原有数组中前n个元素复制到新数组中
  T *srcptr = alist; // 原数组alist的首地址
  T *destptr = newlist; // 新数组newlist的首地址
  while (n--)
    *destptr++ = *srcptr++; // 复制数组元素
  delete[] alist; // 删除原数组
  alist = newlist; // 使alist 指向新数组
  size = sz; // 更新size
}

```

53

```

//获取当前数组的大小
template <class T>
int Array<T>::ListSize(void) const
{
    return size;
}
// 私有函数模板Error实现输出错误信息的功能,
// 仅Array数组类的成员函数可以调用
template <class T>
void Array<T>::Error(ErrorType error, int badIndex) const
{ cout << errorMsg[error]; //根据错误类型,输出相应的错误信息
  if (error == indexOutOfRange)
    cout << badIndex; //如果是下标越界错, 输出错误的下标
  std::cout << endl;
  exit(1);
}

```

54

```

void main(void)
{ Array <int> A(10); // 用来存放质数的数组,开始仅有10个元素
  int n;           //质数范围的上限, 运行时输入
  int primecount = 0, i, j;
  cout << "Enter a value >= 2 as upper limit for prime numbers: ";
  cin >> n;
  A[primecount++] = 2;    // 2是一个质数
  for(i = 3; i < n; i++)
  { // 如果质数表满了, 便再申请多分配10个元素的空间
    if (primecount == A.ListSize( ))
      A.Resize(primecount + 10);
    // 若大于2的偶数不是质数, 则略过本次循环的其余部分,
    // 立刻开始下一次循环
    if (i % 2 == 0)
      continue;
    // 检查3,5,7,...,i/2是否i的因子
    j = 3;

```

55

```

    while (j <= i/2 && i % j != 0)
      j += 2;
    // 若上述参数均不为i的因子,则i为质数
    if (j > i/2)
      A[primecount++] = i;
  }
  for (i = 0; i < primecount; i++)    //输出质数
  { cout << setw(5) << A[i];
    if ((i+1) % 10 == 0)    //每输出10个数换行一次
      cout << endl;
  }
  cout << endl;
  Array <int> B(A); // 用A数组复制构造B数组
  for (i = 0; i < primecount; i++)    //输出质数
  { cout << setw(5) << B[i];
    if ((i+1) % 10 == 0)    //每输出10个数换行一次
      cout << endl;
  }
  cout << endl;

```

56

```

Array <int> C(5);
C = A; // 数组整体赋值
for (i = 0; i < primecount; i++)    //输出质数
{ cout << setw(5) << C[i];
  if ((i+1) % 10 == 0)    //每输出10个数换行一次
    cout << endl;
}
cout << endl;

Array <int> D(5);
*D = *C; // 将C数组第一个元素赋值给D数组第一个元素
cout << "D=" << D[0] << ' ' << D[1] << endl; // D[1]未赋初值

cout << C[100] << endl; // 输出不存在的元素，数组下标越界
}

```

57

运行结果：

Enter a value >= 2 as upper limit for prime numbers: 100

```

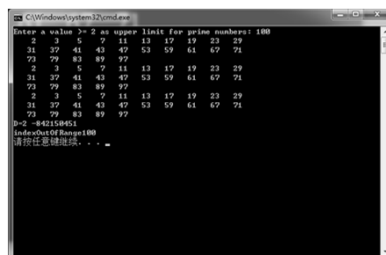
2  3  5  7 11 13 17 19 23 29
31 37 41 43 47 53 59 61 67 71
73 79 83 89 97
2  3  5  7 11 13 17 19 23 29
31 37 41 43 47 53 59 61 67 71
73 79 83 89 97
2  3  5  7 11 13 17 19 23 29
31 37 41 43 47 53 59 61 67 71
73 79 83 89 97

```

D=2 -842150451

Index Out Of Range 100

请按任意键继续...



58

动态多维数组的例子：（用两个类模板）

```
#include <iostream>
using namespace std;
template <class T> class Array;
template <class T> class ArrayBody
{private:
    T *tpBody;
    int iRows, iColumns, iCurrentRow;
public:
    friend class Array<T>;
    ArrayBody(int iRsz, int iCsz)
    {    tpBody = new T[iRsz*iCsz];
        iRows = iRsz; iColumns = iCsz; iCurrentRow = -1;
    }
    T& operator[ ](int j)    // 重载二维数组的第二个[ ]
    { bool row_error, column_error;
      row_error = column_error = false;
      try {
          if (iCurrentRow < 0 || iCurrentRow >= iRows) row_error = true;
          if (j < 0 || j >= iColumns) column_error = true;
          if (row_error == true || column_error == true) throw 'e';
      }
    }
```

59

```
    catch(char)
    {    if (row_error == true)
        cerr << "行下标越界[" << iCurrentRow << "]" ";
        if (column_error == true)
            cerr << "列下标越界[" << j << "]" ";
        cout << endl;
    }
    return tpBody[iCurrentRow * iColumns + j];
}
~ArrayBody( ) { delete [ ]tpBody; }
};
template <class T> class Array
{    private:
    ArrayBody<T> tBody;
    public:
    Array(int iRsz, int iCsz): tBody(iRsz,iCsz){ }
    ArrayBody<T> & operator[ ](int i)// 重载二维数组的第一个[]
    {    tBody.iCurrentRow = i ;
        return tBody;
    }
};
```

60

```
void main( )
{ Array <int> a1(10, 20);
  Array <double> a2(3, 5);
  int b1;
  double b2;
  b1 = a1[-5][10]; // 有越界提示：行下标越界[-5]
  b1 = a1[10][15]; // 有越界提示：行下标越界[10]
  a1[1][4] = 123; // 没有越界提示
  b2 = a2[2][6]; // 有越界提示：列下标越界[6]
  b2 = a2[10][20]; // 有越界提示：行下标越界[10] 列下标越界[20]
  a2[1][4] = 456.789; // 没有越界提示
  cout << a1[1][4] << " " << a2[1][4] << endl;
}
```

运行结果为：

行下标越界[-5]
行下标越界[10]
列下标越界[6]
行下标越界[10] 列下标越界[20]
123 456.789
请按任意键继续...

61

第9次作业：

见网络学堂第9次作业

2题必做，1题选做

62